SQL vs NoSQL

SQL

Definition:

SQL is a standard language used to store, manage, and retrieve data from relational databases. It is mainly designed to work with structured data, where information is stored in the form of tables (rows and columns). SQL databases follow a fixed schema, meaning the structure of the table (columns and their data types) must be defined before inserting data. SQL allows users to perform operations like inserting data, updating records, deleting records, and querying required information efficiently.

Main Features of SQL:

- Works with Structured Data stored in rows and columns
- Fixed Schema Support (table structure must be predefined)
- Data Retrieval using SELECT queries
- Data Insertion using INSERT
- Data Updating using UPDATE
- Data Deletion using DELETE
- Supports Relational Databases like MySQL, Oracle, PostgreSQL, SQL Server
- Supports Joins to combine data from multiple tables
- Filtering and Sorting using WHERE, ORDER BY, GROUP BY, HAVING
- Data Integrity using constraints like PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK

When to Choose SQL:

- Select SQL when the data is organized and should be stored in form of tables (rows and columns).
- It is appropriate when the database cannot be changed in terms of schema and the form is determined in advance and the data is then stored.
- SQL is chosen when the program requires that tables are related, e.g. customer and order information.
- It is preferred in cases where the consistency and accuracy of data are relevant (ACID properties).
- SQL is applicable in cases when CRUD operations are to be made frequently: Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE)
- It is best suited to those requirements that involve complex queries, joins and reports.

Query Language:

SQL databases mainly use SQL language, which is common across most relational databases.

NoSQL

Definition:

NoSQL (Not Only SQL) refers to a kind of database system that stores and manages large amounts of data, which can be structured, semi-structured or unstructured. NoSQL databases do not use fixed tables and most often have a flexible or dynamic schema, unlike SQL databases, which do, with data potentially taking the form of documents, pairs of key-values, wide columns or graphs. NoSQL databases are highly scaled, high-performing, and big data

application friendly, which is why they are appropriate to be used in real-time systems, distributed environments, and applications whose data structure is often changed.

Main Features of NoSQL:

- Flexible Schema: Data structure does not exist and can readily change.
- Supports Unstructured / Semi-Structured Data: Can store JSON, documents, key-value data and so on.
- High Scalability: It is designed to scale horizontally with the addition of additional servers.
- Fast Performance: Asserted to have fast read/write access, particularly with data of big size.
- Distributed Storage: Data can be stored in a number of machines/locations.
- Multiples Data Models: Supports the following models: Document-based (MongoDB), Key-Value (Redis), Column-based (Cassandra), Graph-based (Neo4j)
- Processes Big Data Well: Appropriate in the large scale processing and real-time.
- High Availability: It is applicable to systems that must be up all the time.

When to Choose NoSQL:

- Select NoSQL in cases of the unstructured or semi-structured data, like the JSON, documents, logs, or multimedia data.
- It can be used with big data and real-time applications such as social media, IoT, and analytics.
- Select NoSQL in cases where the system requires rapid read/write and where large amount of data is required.

- It is adaptable to distributed applications that have a need of high availability and replication.

Query Language:

The NoSQL databases do not share a single query language, such as SQL. Every NoSQL database has its query language or commands, which include: MQL in MongoDB, CQL in Cassandra, Redis commands in Redis, and Cypher in Neo4j.

Difference Between SQL and NoSQL

Aspect	SQL	NoSQL
Data Structure	Tables with rows and columns	Document-based, key-value, column-family, or graph-based
Schema	Fixed Schema (predefined structure)	Flexible Schema (dynamic and adaptable)
Data Integrity	ACID-compliant (strong consistency)	BASE-compliant (more available, less consistent)
Query Language	SQL (Structured Query Language)	Varies (e.g. MongoDB uses its own query language)
Performance	Efficient for complex queries and transactions	Efficient for complex queries and transactions
Use Case	Best for transactional systems (banking, ERP, etc.)	Ideal for big data, real-time web apps, and data lakes
Examples	MySQL, PostgreSQL, Oracle, MS SQL Server	MongoDB, Cassandra, CouchDB, Neo4j